

Guion de Presentación

Sistema de Facturación y Cuentas por Cobrar / Pagar

C# · Windows Forms · ADO.NET · SQL Server | Arquitectura de 3 capas | Login demo: `admin` / `admin123`

Formato sugerido: video explicativo en grupo (~12–15 min). Cada persona comparte pantalla con sus archivos y con la aplicación corriendo. La división sigue la **arquitectura de 3 capas** del proyecto, de modo que cada quien explica una parte coherente y sin solaparse.

Persona 1 (Presentación) → Persona 2 (Negocio) → Persona 3 (Datos)

PERSONA 1 Arquitectura y Presentación (Front-end)

Rol: presenta el proyecto, explica la arquitectura general y la capa que ve el usuario (formularios Windows Forms). Hace la demo en vivo.

Archivo	Qué mostrar
README.md · Informe.md	De qué trata el sistema y sus módulos
Program.cs	Punto de entrada + manejador global de errores
App.config	Dónde se configura la conexión (solo mencionar)
Presentacion/FrmLogin.cs	Pantalla de inicio de sesión
Presentacion/FrmPrincipal.cs	Menú principal y navegación
Presentacion/FrmFacturaNueva.cs	Formulario que arma una factura
Presentacion/UI.cs	Utilidades comunes de la interfaz

Qué decir

"Buenas, somos [nombres] y este es nuestro **Sistema de Facturación y Cuentas por Cobrar y por Pagar**, una aplicación de escritorio hecha en **C# con Windows Forms** y base de datos **SQL Server**.

El sistema está organizado en **3 capas separadas**: la de **Presentación**, que es lo que ve el usuario; la de **Negocio**, con las reglas y cálculos; y la de **Acceso a Datos**, que habla con la base de datos. Yo explico la de Presentación; mis compañeros, las otras dos.

Todo arranca en `Program.cs`, el punto de entrada: abre el login e instala un **manejador global de excepciones** para que, si algo falla, la app no se cierre de golpe sino que muestre un mensaje.

La primera pantalla es `FrmLogin`. Fíjense que la Presentación **no sabe nada de SQL**: solo pasa los datos a la capa de Negocio y espera la respuesta. Una vez dentro, `FrmPrincipal` es el menú central: Clientes, Productos, Facturas, Cobros, Compras y Pagos. El módulo de Usuarios solo aparece si entramos como **administrador**.

El formulario estrella es `FrmFacturaNueva`: elijo el cliente, agrego productos con su cantidad, y en pantalla se ven en vivo el **subtotal, el ITBIS (18%) y el total**. Al guardar, este formulario no calcula ni graba nada por su cuenta: le entrega la factura a la capa de Negocio, que explica mi compañero/a."

 **Demo:** hacer login, entrar al menú, crear una factura de ejemplo y mostrar cómo se actualiza el total. Luego pasar el turno.

PERSONA 2 Lógica de Negocio (reglas y cálculos)

Rol: explica el "cerebro" del sistema — cálculos, validaciones, saldos y seguridad. La capa que decide qué se puede hacer y qué no.

Archivo	Qué mostrar
Entidades/*.cs (Factura, Cliente, Producto...)	Las clases-objeto que representan el negocio
Negocio/FacturaNegocio.cs	Cálculo de factura + validaciones (ejemplo estrella)
Negocio/Validaciones.cs	Validaciones de entrada (campos, números, email)
Negocio/Seguridad.cs	Cifrado de contraseñas con SHA-256
Negocio/CobroNegocio.cs · CompraNegocio.cs · PagoNegocio.cs	Cuentas por cobrar y por pagar
Negocio/Sesion.cs · Configuracion.cs	Usuario activo y tasa de impuesto (18%)

Qué decir

"Gracias. Yo explico la **capa de Negocio**, donde viven las reglas del sistema.

En la carpeta `Entidades` tenemos clases como `Factura`, `Cliente` o `Producto`. Son objetos que representan las cosas reales del negocio — una `Factura` tiene su subtotal, impuesto, total y su lista de detalles. Esto es **programación orientada a objetos**: encapsulamos los datos en objetos.

El archivo clave es `FacturaNegocio.cs`. Cuando la pantalla pide crear una factura, aquí pasa todo lo importante:

- En `CalcularTotales` se suma el importe de cada línea para el **subtotal**, se calcula el **ITBIS del 18%** (definido en `Configuracion.cs`) y se obtiene el **total**.
- Según el tipo de pago se decide el saldo: **Contado** → factura pagada, saldo 0; **Crédito** → pendiente, con saldo igual al total (una **cuenta por cobrar**).
- Valida las reglas: que haya cliente, que haya al menos un producto, que **haya stock suficiente** y que la venta a crédito **no supere el límite de crédito**. Si algo falla, lanza un `NegocioException` con mensaje amigable.

En `Validaciones.cs` están las validaciones de entrada: campos obligatorios, montos positivos, formato de correo. Y en `Seguridad.cs`, algo importante: las contraseñas **nunca se guardan en texto plano**; se cifran con **SHA-256** y al iniciar sesión comparamos el hash, no la contraseña real.

Cobros, Compras y Pagos funcionan igual: cada cobro reduce el saldo del cliente, cada compra aumenta lo que debemos al proveedor y cada pago lo reduce. Pero *cómo* se graba eso de forma segura lo explica mi compañero/a."

PERSONA 3 Base de Datos y Acceso a Datos (ADO.NET)

Rol: explica dónde y cómo se guardan los datos — el diseño de la base de datos y la capa que ejecuta el SQL con transacciones.

Archivo	Qué mostrar
Database/01_CrearBaseDatos.sql	Tablas, llaves primarias/foráneas, restricciones
Database/02_DatosPrueba.sql	Datos iniciales de prueba
DatosAcceso/ConexionBD.cs	Conexión central + protección contra inyección SQL
DatosAcceso/FacturaDAO.cs	Transacción al crear/anular una factura (ejemplo estrella)
DatosAcceso/*.cs (los otros DAO)	Cada entidad tiene su propio DAO

Qué decir

"Y para cerrar, yo explico dónde termina viviendo todo: la **base de datos** y la **capa de Acceso a Datos**.

En `Database/01_CrearBaseDatos.sql` se crea la base `SistemaFacturacionDB` con todas sus tablas: Usuarios, Clientes, Proveedores, Productos, Facturas, DetalleFactura, Cobros, Compras, DetalleCompra y Pagos. Está **normalizada** y con integridad referencial: **llaves primarias**, **llaves foráneas**, y restricciones **UNIQUE**, **CHECK** y **DEFAULT** para que no entren datos inválidos. El script `02_DatosPrueba.sql` carga datos de ejemplo para poder probar.

La puerta de entrada es `ConexionBD.cs`: lee la cadena de conexión del `App.config` y ofrece métodos reutilizables para ejecutar consultas. Lo más importante: **todo se hace con parámetros** (`@cliente`, `@monto` ...) en lugar de pegar texto — esa es la defensa principal contra la **inyección SQL**.

Cada entidad tiene su **DAO** — por ejemplo `FacturaDAO.cs` — que traduce entre los objetos de C# y las tablas de SQL. Lo más interesante está en `FacturaDAO.Insertar`: crear una factura toca **cuatro cosas a la vez** — el encabezado, las líneas de detalle, el **descuento del inventario** y el **saldo del cliente**. Todo dentro de una **transacción** (`SqlTransaction`): o se confirman todos los pasos con `Commit`, o si algo falla se deshace todo con `Rollback`. Así nunca queda una factura sin detalle o un stock descuadrado.

Lo mismo al **anular** una factura: se devuelve el inventario, se baja el saldo del cliente y se marca como anulada — otra vez, todo en una transacción para mantener las cuentas cuadradas.

Y así trabajan juntas las tres capas: la Presentación pide, el Negocio decide y el Acceso a Datos guarda de forma segura. Gracias por ver nuestro proyecto."

Reparto por capas

Persona	Capa	Carpets principales
1 — Presentación	Front-end	Presentacion/ , Program.cs , App.config
2 — Negocio	Reglas y cálculos	Negocio/ , Entidades/
3 — Datos + BD	Persistencia	DatosAcceso/ , Database/

Consejos para el video

- **Sigan el flujo de una misma factura** entre los tres: Persona 1 la crea en pantalla → Persona 2 explica cómo se calcula y valida → Persona 3 muestra cómo se guarda con transacción. Da una narrativa continua.
- Cada quien con **su archivo abierto** al hablar (pantalla compartida).
- Frase que suma puntos: *"la Presentación no conoce SQL, la lógica está aislada, y si cambiáramos de base de datos solo tocaríamos una capa"* — demuestra que entienden la arquitectura.